

IF2211 ALGORITHM STRATEGY

Pattern Matching in Chromium Browser Extension for Online Gambling Detection

May 29, 2026



Made Branenda Jordhy - 13524026

Stefani Angeline Oroh - 13524064

Michael James Liman - 13524106

School of Electrical Engineering and Informatics
Institut Teknologi Bandung
Semester II 2025/2026

Contents

1 Overview	3
2 Landasan Teori	4
2.1 Algoritma Knuth-Morris-Pratt (KMP)	4
2.2 Algoritma Boyer-Moore (BM)	5
2.3 Algoritma Aho-Corasick (Bonus)	6
2.4 Algoritma Rabin-Karp (Bonus)	7
2.5 Weighted Levenshtein Distance	7
2.6 Regular Expression	8
2.7 Chromium Browser Extension	8
3 Problem Analysis & Solution Design	10
4 Implementation & Testing	11
5 Conclusion & Suggestion	12
Appendix	14

1 Overview

Proyek ini kami kerjakan untuk memenuhi Tugas Besar 3 mata kuliah IF2211 Strategi Algoritma. Topiknya sudah ditentukan, yaitu membuat sebuah *Chromium browser extension* yang bisa mendeteksi konten judi online di halaman web, sekaligus jadi tempat menerapkan algoritma pencocokan string yang dipelajari di kelas.

Topik judol sendiri sebetulnya relevan jika dilihat dari kondisi sekarang. Iklannya ada di mana-mana, mulai dari hasil pencarian Google, kolom komentar YouTube, sampai sisipan iklan di situs yang sebenarnya tidak ada hubungannya sama sekali dengan perjudian. Nama-nama yang dipakai pun mirip semua, biasanya berpola <kata><angka> seperti GACOR99, MAXWIN88, atau MADU308, supaya gampang diingat dan susah disaring otomatis. Tidak jarang pelakunya juga menyamarkan tulisan dengan karakter yang bentuknya mirip, misalnya huruf O jadi angka 0, A jadi 4, atau I jadi 1, sehingga filter yang cuma membandingkan string mentah sering kebobolan.

Ekstensi yang kami bangun untuk tugas ini kami beri nama **Judol Detector**. Begitu halaman selesai dimuat, ekstensi akan menelusuri DOM, mencocokkan teks dengan daftar kata kunci, lalu menandai elemen yang mengandung kemunculan kata kunci tersebut. Pengguna bisa mengarahkan kursor ke teks yang ditandai untuk melihat detailnya, atau membuka panel popup ekstensi untuk melihat ringkasan statistik secara keseluruhan. Pencocokan kata kunci dijalankan dengan beberapa algoritma sekaligus, yaitu Knuth-Morris-Pratt dan Boyer-Moore untuk tahap pencocokan eksak, RegEx untuk menangkap pola <kata><angka> yang belum tentu ada di daftar kata kunci, lalu Weighted Levenshtein Distance untuk tahap fuzzy yang menangani kasus penyamaran karakter. Seluruh algoritma string matching dibuat *from scratch*, tanpa bantuan `String.includes()`, `String.indexOf()`, atau pustaka pencocokan string apapun. Tiap algoritma juga menghitung jumlah perbandingan karakter (*comparison counting*) supaya performanya bisa dibandingkan secara objektif.

2 Landasan Teori

2.1 Algoritma Knuth-Morris-Pratt (KMP)

KMP pada dasarnya merupakan satu observasi sederhana pada matching brute-force. Jika pencarian gagal di tengah jalan, sebetulnya kita sudah punya cukup informasi dari karakter yang sudah dibandingkan untuk tahu sejauh mana pola harus digeser. Brute-force tidak peduli soal itu dan mengembalikan pointer teks ke posisi sebelumnya, jadi banyak perbandingan yang sudah dilakukan terbuang sia-sia. KMP memanfaatkan informasi tadi supaya pointer teks bisa terus maju. Hasilnya, kompleksitas pencariannya $O(n + m)$ untuk teks berukuran n dan pola berukuran m .

Kunci algoritma ini adalah *border function*, yaitu tabel yang menyimpan panjang *proper prefix* terpanjang pada pola yang sekaligus juga merupakan suffix-nya, sampai posisi tertentu. Tabel ini dihitung sekali di awal dengan kompleksitas $O(m)$. Kalau mismatch terjadi di indeks i pada pola, pointer pola dipindahkan ke posisi $b[i - 1]$, sementara pointer teks tetap di tempatnya. Contoh nilai border function untuk pola **ababaca** bisa dilihat di Tabel 1.

Table 1: Border function untuk pola **ababaca**.

i	0	1	2	3	4	5	6
$P[i]$	a	b	a	b	a	c	a
$b[i]$	0	0	1	2	3	0	1

Kompleksitas pencariannya sebetulnya linier baik di kasus terbaik maupun terburuk, cara mencapainya saja yang berbeda [2]. Misalkan pola berukuran m dicari pada teks berukuran n . Kasus paling sederhana terjadi saat pola langsung ditemukan di awal teks dengan biaya $O(m)$.

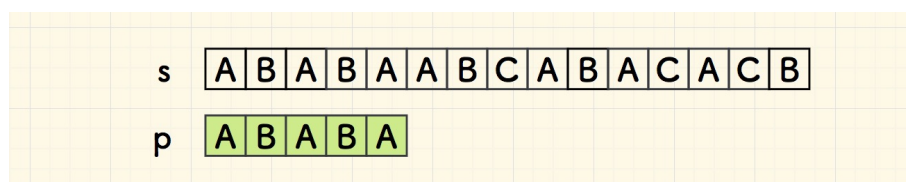


Figure 1: Kasus terbaik trivial: pola langsung ditemukan di awal teks (source: [stackoverflow](#)).

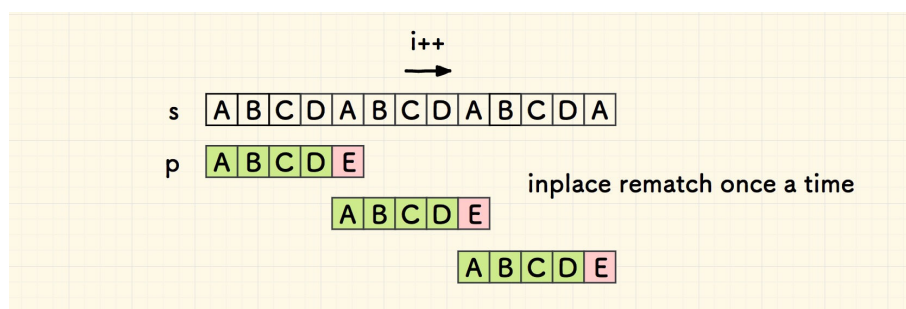


Figure 2: Kasus terbaik saat pola tidak ada di teks: tiap mismatch jatuh di karakter pertama pola (source: [stackoverflow](#)).

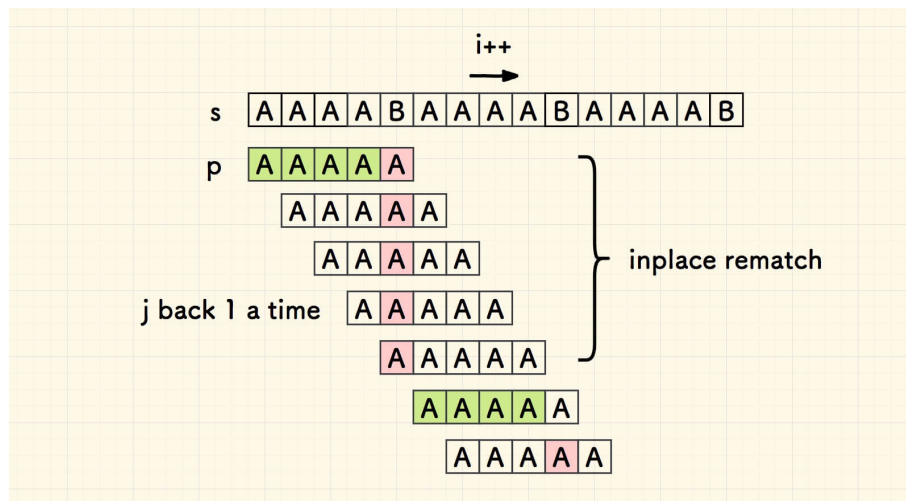


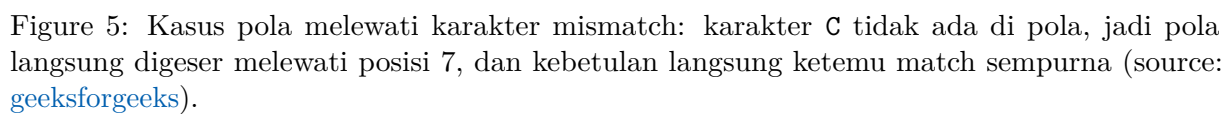
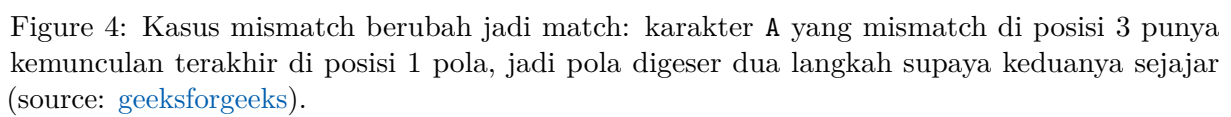
Figure 3: Kasus terburuk: mismatch baru terjadi setelah hampir seluruh karakter pola dibandingkan (source: [stackoverflow](#)).

Di implementasi kami, perbandingan saat membangun border function juga ikut dihitung sebagai *comparison count*, bukan cuma yang terjadi saat pencarian. Angka inilah yang nantinya dipakai untuk membandingkan performa antar algoritma di Bab 4.

2.2 Algoritma Boyer-Moore (BM)

Ciri utama Boyer-Moore adalah arah perbandingannya. Pointer pola tetap maju dari kiri ke kanan terhadap teks, tapi karakter pola dibandingkan dari kanan ke kiri. Cara ini memungkinkan pola melompat banyak karakter sekaligus saat mismatch, dan praktiknya sering lebih cepat dari KMP terutama untuk pola yang panjang. Boyer-Moore standar punya dua heuristik, *bad character* dan *good suffix*. Yang kami pakai di sini adalah *bad character* via **last occurrence table**, yaitu tabel posisi kemunculan terakhir tiap karakter pada pola.

Saat mismatch terjadi, ada dua kemungkinan yang ditangani heuristik ini [3]. Pertama, kalau karakter mismatch masih ada di pola, pola digeser supaya kemunculan terakhir karakter tersebut sejajar dengan posisi mismatch di teks (Figure 4). Kedua, kalau karakter mismatch tidak ada di pola sama sekali, pola langsung digeser melewati posisi mismatch tersebut (Figure 5).



2.3 Algoritma Aho-Corasick (Bonus)

Failure link di sini mirip dengan border function pada KMP. Kalau mismatch terjadi di sebuah simpul, automaton tidak balik ke akar, tapi melompat ke simpul terdalam yang merepresentasikan *proper suffix* terpanjang dari string yang sudah dicocokkan sejauh ini [4]. Contoh struktur automatonnya dapat dilihat pada Gambar 6.

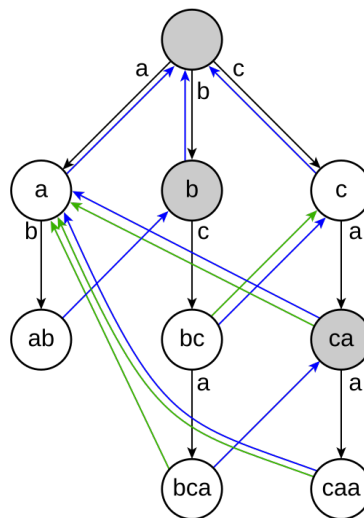


Figure 6: Contoh struktur automaton Aho-Corasick. Panah hitam adalah *goto transition* dan panah biru adalah *failure link* yang menunjuk ke simpul terdalam yang merepresentasikan *proper suffix* terpanjang dari string yang sudah dicocokkan (sumber: [wikipedia](https://en.wikipedia.org/wiki/Aho%E2%80%93Corasick_algorithm)).

Untuk Judol Detector, daftar kata kunci judul bisa cukup panjang. Dengan KMP atau BM, teks harus diulang sebanyak jumlah kata kunci. Aho-Corasick cukup sekali jalan.

2.4 Algoritma Rabin-Karp (Bonus)

Rabin-Karp memakai pendekatan berbasis *hashing*. Daripada membandingkan karakter satu per satu untuk tiap posisi pada teks, algoritma ini menghitung nilai hash dari setiap *window* berukuran m di teks, lalu membandingkannya dengan hash pola. Cuma kalau hash-nya cocok, baru dilakukan pengecekan karakter penuh untuk memastikan bukan tabrakan hash (*spurious hit*).

Yang membuat Rabin-Karp tetap efisien adalah **rolling hash**, biasanya Rabin fingerprint, yang memungkinkan perhitungan hash window berikutnya cuma $O(1)$ dari hash window sebelumnya. Kompleksitas rata-ratanya $O(n + m)$, walau worst-case-nya tetap $O(nm)$ kalau hash sering bertabrakan. Rabin-Karp juga cocok untuk pencocokan multi-pola karena hash banyak pola bisa disimpan di tabel dan diuji bersamaan terhadap *rolling hash* teks.

2.5 Weighted Levenshtein Distance

Levenshtein distance adalah jumlah operasi minimum (*insert*, *delete*, *substitute*) untuk mengubah satu string jadi string lain. Versi standarnya memberi bobot yang sama untuk semua operasi, biasanya 1, dan dihitung pakai *dynamic programming* dalam waktu $O(nm)$.

Pada varian *Weighted Levenshtein*, bobot tiap operasi bisa disetel berbeda-beda. Di kasus judul, substitusi antar karakter yang mirip secara visual (O dengan 0, I dengan 1, A dengan 4) diberi bobot lebih kecil daripada substitusi karakter yang benar-benar berbeda. Jadi penyamaran semacam HOKI88 versus H0KI88 tetap dianggap mirip, sementara substitusi acak akan dihukum penuh. Tabel 2 mengilustrasikan perhitungannya untuk pasangan OKI dan OKI.

Table 2: Matriks DP Weighted Levenshtein untuk mengubah OKI menjadi OKI, dengan bobot substitusi $O \leftrightarrow 0$ sebesar 0.3 dan operasi lain bernilai 1. Sel pojok kanan bawah (cetak tebal) adalah jaraknya, yaitu 0.3. Dengan bobot standar (semua substitusi bernilai 1), nilainya akan jadi 1, sehingga ambang batas kemiripan biasanya tidak tercapai.

	ε	O	K	I
ε	0	1	2	3
O	1	0.3	1.3	2.3
K	2	1.3	0.3	1.3
I	3	2.3	1.3	0.3

Untuk membatasi sampai mana dua string masih dianggap cocok, dipakai ambang batas (*threshold*) pada skor kemiripan ($1 - \text{normalized distance}$). Perhitungan juga dipecah per token (split berdasarkan spasi) supaya teks panjang tidak salah cocok karena *distance* menumpuk dari banyak kata sekaligus.

2.6 Regular Expression

Regular expression (Regex) adalah bahasa untuk mendefinisikan pola tekstual secara formal. Mesin Regex modern, termasuk yang ada di JavaScript, biasanya berbasis NFA dengan *backtracking*. Di sini, Regex kami pakai untuk menangkap pola `<kata><angka>` yang sering jadi ciri khas nama situs judol, yaitu kombinasi minimal dua huruf diikuti minimal dua angka, dibatasi oleh *word boundary*.

Beda dari KMP atau BM yang butuh daftar kata kunci yang sudah diketahui, Regex berperan sebagai *discovery layer*. Kalau ada nama situs baru yang belum masuk daftar kata kunci tapi tetap mengikuti pola tersebut, ekstensi tetap bisa mendeteksinya.

2.7 Chromium Browser Extension

Chromium browser extension adalah perangkat lunak kecil yang dipasang di browser berbasis Chromium (Chrome, Edge, Brave, dan sejenisnya), yang bisa menambah atau memodifikasi fungsi browser saat menjelajah web. Sejak **Manifest V3**, sebuah ekstensi umumnya terdiri dari beberapa komponen utama.

- **Manifest** (`manifest.json`), berkas konfigurasi yang mendeklarasikan nama, versi, izin (*permissions*), dan daftar komponen ekstensi.
- **Content script**, skrip yang disuntikkan langsung ke halaman web dan bisa berinteraksi dengan DOM. Inilah komponen yang menjalankan pemindaian teks pada Judol Detector.
- **Popup**, halaman kecil yang muncul saat ikon ekstensi diklik, biasanya berisi UI untuk menampilkan ringkasan informasi dan tombol-tombol kontrol.
- **Service worker** (background), skrip latar yang menangani *event* global dan menyimpan *state* ekstensi. Pada Manifest V3, service worker hanya hidup saat dibutuhkan, berbeda dengan background page Manifest V2 yang persisten.

Ketiga komponen ini berkomunikasi lewat API `chrome.runtime`, biasanya pakai pesan (`sendMessage` dan `onMessage`) atau lewat `chrome.storage` kalau perlu berbagi *state*. Arsitektur Judol Detector secara umum tergambar pada Gambar 7.

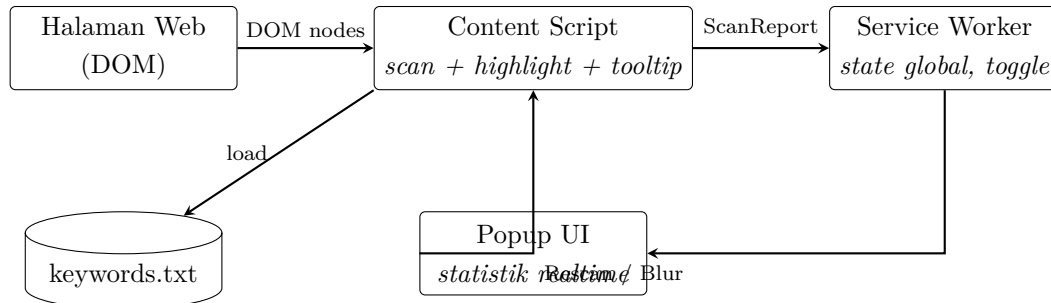


Figure 7: Arsitektur Judol Detector. *Content script* adalah pemain utama yang menyentuh DOM dan menjalankan algoritma pencocokan, sementara popup dan *service worker* berperan sebagai pengontrol dan penampung *state*.

Pada Judol Detector, *content script* bertugas menelusuri DOM dan menjalankan algoritma pencocokan, popup menampilkan statistik *realtime* hasil pemindaian beserta kontrol *blur toggle* dan *rescan*, sedangkan *service worker* memelihara *state* global seperti status *blur* antar tab.

3 Problem Analysis & Solution Design

4 Implementation & Testing

5 Conclusion & Suggestion

References

- [1] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Addison-Wesley, 2006.
- [2] “What is the best/worst time complexity of KMP algorithm?,” Stack Overflow, Sep. 2020. [Online]. Available: <https://stackoverflow.com/questions/63867161/what-is-the-best-worst-time-complexity-of-kmp-algorithm>
- [3] “Boyer-Moore Algorithm for Pattern Searching,” GeeksforGeeks. [Online]. Available: <https://www.geeksforgeeks.org/dsa/boyer-moore-algorithm-for-pattern-searching/>
- [4] “Aho-Corasick algorithm,” Wikipedia. [Online]. Available: https://en.wikipedia.org/wiki/Aho-Corasick_algorithm

Appendices

Statement

GitHub Repository

https://github.com/ethj0r/Tubes3_LeGamblers

Video Link

Spesification Checklist

Task Distribution